

Kubernetes interview & Production Issues Q&A

06/05/2026 08:26:43

<https://claude.ai/chat/1f83f318-79d6-4dff-a378-8e5ec1876660>

All Kubernetes interview questions

Here's a comprehensive set of Kubernetes interview questions organized by...

Answers these above questions ALONG with that after completing all these...

This is a large comprehensive guide. Let me deliver it in structured sections. ...

Any other issues we can expect from Kubernetes based application and how...

Here are additional real-world production issues you can expect from...



Here's a comprehensive set of Kubernetes interview questions organized by topic and difficulty.

Core Concepts

Basic

- What is Kubernetes and why is it used?
- What are the main components of a Kubernetes cluster?
- What is the difference between a Pod, Deployment, and ReplicaSet?
- What is a Namespace and when would you use it?
- What is the role of etcd in a Kubernetes cluster?

Intermediate

- What is the difference between a StatefulSet and a Deployment?
 - When would you use a DaemonSet vs a Deployment?
 - What is a Job vs a CronJob?
 - What is the difference between a ConfigMap and a Secret?
 - What is an Init Container and when is it useful?
-

Architecture

- Explain the Kubernetes control plane components (API Server, Scheduler, Controller Manager, etcd).
- What is the role of the kubelet and kube-proxy on a worker node?
- How does the Kubernetes API Server handle requests?

- What is the Controller Manager and what controllers does it run?
 - How does the Scheduler decide which node to place a Pod on?
 - What is the Cloud Controller Manager?
-

Networking

- What is a ClusterIP, NodePort, and LoadBalancer service? When do you use each?
 - What is an Ingress and how does it differ from a Service?
 - How does DNS work inside a Kubernetes cluster?
 - What is a NetworkPolicy and how do you restrict traffic between Pods?
 - What is the CNI (Container Network Interface) and name some CNI plugins.
 - How does Pod-to-Pod communication work across nodes?
 - What is a headless service and when would you use it?
-

Storage

- What is a PersistentVolume (PV) and PersistentVolumeClaim (PVC)?
 - What is a StorageClass and what is dynamic provisioning?
 - What is the difference between ReadWriteOnce, ReadOnlyMany, and ReadWriteMany access modes?
 - What happens to a PV when the PVC is deleted? (Reclaim policies)
 - What is a VolumeMount vs a Volume in a Pod spec?
 - How would you share storage between containers in the same Pod?
-

Scheduling & Resource Management

- What are resource requests and limits? Why do they matter?
- What is a LimitRange and ResourceQuota?
- What are taints and tolerations? Give a real-world use case.
- What are node affinity and pod affinity/anti-affinity rules?
- What is a PriorityClass and how does Pod preemption work?
- What is the difference between Guaranteed, Burstable, and BestEffort QoS classes?

Scaling & Availability

- How does the Horizontal Pod Autoscaler (HPA) work?
- How does the Vertical Pod Autoscaler (VPA) work? How does it differ from HPA?
- What is the Cluster Autoscaler?
- What is a PodDisruptionBudget (PDB) and why is it important?
- What is a rolling update strategy vs recreate strategy in a Deployment?
- How do you implement blue-green or canary deployments in Kubernetes?

Security

- What is RBAC in Kubernetes? Explain Role, ClusterRole, RoleBinding, ClusterRoleBinding.
- What is a ServiceAccount and when would a Pod need one?
- What is a PodSecurityContext vs a container SecurityContext?
- What are Pod Security Standards (restricted, baseline, privileged)?
- How do you store and manage sensitive data securely in Kubernetes?
- What is the principle of least privilege in Kubernetes RBAC?
- What is a NetworkPolicy and how does it enforce zero-trust networking?
- How do you scan container images for vulnerabilities in a CI/CD pipeline?

Observability

- How do you view logs for a Pod? For a previous crashed container?
- What is the difference between liveness, readiness, and startup probes?
- How do you monitor a Kubernetes cluster? (Prometheus, Grafana stack)
- What metrics does the Kubernetes Metrics Server expose?
- How does distributed tracing work in a microservices Kubernetes environment?
- What is the difference between `kubectl logs` and a centralized logging solution?

Configuration & Helm

- What is Helm and what problem does it solve?
 - What is the difference between a Helm chart, release, and repository?
 - What is `values.yaml` and how do you override values?
 - What is Helm's `--dry-run` flag used for?
 - What is the difference between `helm upgrade` and `helm rollback`?
 - What are Helm hooks and when would you use them?
-

CI/CD & GitOps

- How do you implement CI/CD pipelines with Kubernetes?
 - What is GitOps and how does it differ from traditional CD?
 - What is ArgoCD? How does it implement GitOps?
 - What is the difference between push-based and pull-based deployment models?
 - How do you handle secrets in a GitOps workflow?
-

Troubleshooting

- A Pod is stuck in `Pending` state. How do you debug it?
 - A Pod is in `CrashLoopBackOff`. What are the possible causes and how do you fix it?
 - A Pod is `Running` but not receiving traffic. How do you investigate?
 - A node shows `NotReady`. What steps do you take?
 - How do you debug a failing Init Container?
 - How do you `exec` into a running container for debugging?
 - A Deployment rollout is stuck. How do you diagnose and fix it?
-

Advanced / Scenario-Based

- How would you design a multi-tenant Kubernetes cluster?
- How do you implement zero-downtime deployments?
- How would you migrate a stateful application (e.g., MySQL) to Kubernetes?
- What is an Operator pattern? When would you write a custom operator?

- What is a Custom Resource Definition (CRD)?
 - How does Kubernetes handle node failure and Pod rescheduling?
 - How would you set up a multi-region or multi-cluster Kubernetes architecture?
 - What is service mesh (Istio/Linkerd) and when do you need it?
 - How do you handle secret rotation without Pod restarts?
 - How would you implement network isolation between teams sharing a cluster?
-

kubectl Commands (Practical)

- How do you force delete a stuck Pod?
 - How do you apply a manifest vs create vs replace?
 - How do you get all resources across all namespaces?
 - How do you debug with a temporary debug pod?
 - How do you perform a rollback of a Deployment?
 - What does `kubectl explain` do?
 - How do you edit a live resource with `kubectl edit`?
 - How do you drain a node before maintenance?
-

Want me to go deep on any specific section with detailed answers, scenario walkthroughs, or follow-up Q&A pairs?

6/5/2026, 6:23:19 AM



Answers these above questions
ALONG with that after completing all these interview questions add
q&a on production issues and how we should fix in Kubernetes or
how we analyze in Kubernetes

6/5/2026, 6:24:41 AM



This is a large comprehensive guide. Let me deliver it in structured sections.

CORE CONCEPTS

What is Kubernetes and why is it used?

Kubernetes (K8s) is an open-source container orchestration platform that automates deploying, scaling, and managing containerized applications. You use it because manually managing containers across many servers is error-prone and unscalable. It gives you self-healing, auto-scaling, service discovery, rolling updates, and declarative configuration out of the box.

What are the main components of a Kubernetes cluster?

A cluster has two sides — the control plane and worker nodes.

Control plane: API Server (entry point for all commands), etcd (cluster state store), Scheduler (assigns Pods to nodes), Controller Manager (runs reconciliation loops), and optionally Cloud Controller Manager.

Worker nodes: kubelet (agent that manages Pods on the node), kube-proxy (handles networking rules), and the container runtime (containerd, CRI-O).

What is the difference between a Pod, Deployment, and ReplicaSet?

A Pod is the smallest deployable unit — one or more containers that share network and storage. It has no self-healing.

A ReplicaSet ensures a specified number of Pod replicas are running at all times. If a Pod dies, it creates a new one.

A Deployment wraps a ReplicaSet and adds rollout management — rolling updates, rollbacks, and versioning. In practice you always create Deployments, never raw ReplicaSets.

What is a Namespace and when would you use it?

A Namespace is a virtual cluster inside a physical cluster. It provides scope for names, RBAC boundaries, and resource quotas. You use it to separate environments (dev/staging/prod) on the same cluster, or to isolate teams in a multi-tenant setup.

What is the role of etcd in a Kubernetes cluster?

etcd is a distributed key-value store that holds the entire cluster state — all object definitions, secrets, config, and metadata. It is the single source of truth. If etcd goes down, the control plane cannot function. This is why production clusters run etcd with 3 or 5 replicas for high availability and take regular backups.

What is the difference between a StatefulSet and a Deployment?

A Deployment is for stateless workloads. Pods are interchangeable, have random names, and can be replaced freely.

A StatefulSet is for stateful workloads like databases. It gives each Pod a stable, ordered name (pod-0, pod-1), a stable DNS identity, and a dedicated PersistentVolume that follows the Pod. Scaling and updates happen in order. Use it for Kafka, Zookeeper, MySQL, Cassandra, etc.

When would you use a DaemonSet vs a Deployment?

A DaemonSet ensures one Pod runs on every node (or a selected subset). Use it for node-level infrastructure — log collectors (Fluentd), metrics agents (Prometheus node-exporter), security agents, CNI plugins. A Deployment, by contrast, runs a specified number of replicas spread across nodes for your application workloads.

What is a Job vs a CronJob?

A Job runs one or more Pods to completion. When the task finishes successfully, the Job is done. Use it for batch processing, database migrations, data exports.

A CronJob is a Job with a schedule expression (standard cron format). It creates Jobs on a time-based trigger — like running a report every midnight or purging old records weekly.

What is the difference between a ConfigMap and a Secret?

Both inject configuration into Pods. A ConfigMap holds non-sensitive data — app config, feature flags, environment-specific properties. A Secret holds sensitive data — passwords, tokens, TLS certs — and is base64-encoded at rest (and can be encrypted with KMS in production).

Secrets should never be committed to Git in plain form. In production, use external secret managers like AWS Secrets Manager, HashiCorp Vault, or Sealed Secrets with GitOps.

What is an Init Container and when is it useful?

An Init Container runs to completion before any app container starts. It is used for pre-flight checks — waiting for a database to be ready, pre-populating a volume with config files, running schema migrations, or fetching secrets. If the Init Container fails, the Pod restarts it. The app container only starts after all Init Containers succeed.

ARCHITECTURE

Explain the Kubernetes control plane components.

API Server: The frontend of the control plane. All kubectl commands, internal components, and external tools communicate through it via REST. It validates and persists state to etcd.

Scheduler: Watches for unscheduled Pods and assigns them to nodes based on resource availability, taints, affinity rules, and policies.

Controller Manager: Runs a collection of controllers — Node Controller (monitors node health), ReplicaSet Controller (maintains desired replica count), Endpoints Controller, and many others. Each controller watches the API Server and reconciles actual state with desired state.

etcd: Stores all cluster state as key-value pairs. Only the API Server communicates directly with etcd.

What is the role of kubelet and kube-proxy on a worker node?

kubelet is an agent running on each node. It watches the API Server for Pods assigned to its node and instructs the container runtime (containerd) to start, stop, or restart containers. It also runs health checks (liveness/readiness probes) and reports Pod status back.

kube-proxy runs on each node and maintains iptables or IPVS rules that implement Kubernetes Service networking — routing traffic to the correct Pod endpoints.

How does the Kubernetes Scheduler decide which node to place a Pod on?

Scheduling is a two-phase process.

Filtering: Eliminates nodes that do not meet the Pod's requirements — insufficient CPU/memory requests, taints without matching tolerations, nodeSelector mismatch, volume zone constraints, etc.

Scoring: Ranks the remaining nodes using priority functions — most available resources, spreading Pods evenly, topology spread, affinity preferences.

The Pod is assigned to the highest-scoring node.

What is the Cloud Controller Manager?

It is the bridge between Kubernetes and the underlying cloud provider (AWS, GCP, Azure). It manages cloud-specific resources — provisioning LoadBalancer services (ELBs), attaching cloud volumes (EBS), managing node lifecycle when cloud VMs are terminated, and updating node metadata with cloud labels. It allows Kubernetes core to remain cloud-agnostic.

NETWORKING

What is ClusterIP, NodePort, and LoadBalancer?

ClusterIP (default): Exposes the service on an internal cluster IP. Only accessible within the cluster. Used for internal microservice communication.

NodePort: Exposes the service on a static port on every node's IP (range 30000-32767). Accessible from outside the cluster via nodeIP:nodePort. Used for dev/test, not production.

LoadBalancer: Provisions a cloud load balancer (e.g., AWS ALB/NLB) that routes external traffic to the service. The standard way to expose services to the internet in production.

What is an Ingress and how does it differ from a Service?

A Service exposes one application at one IP/port. An Ingress is an L7 HTTP/HTTPS router that can route traffic to multiple services based on host and path rules, handle TLS termination, and apply rate limiting or auth — all from a single external IP. An Ingress Controller (Nginx, AWS ALB Ingress, Traefik) implements the Ingress rules.

For example: `api.example.com/claims` routes to the claims-service,
`api.example.com/auth` routes to the auth-service.

How does DNS work inside a Kubernetes cluster?

CoreDNS runs as a Deployment in the `kube-system` namespace and acts as the cluster DNS server. Every Service gets a DNS entry in the format `<service>.<namespace>.svc.cluster.local`. Every Pod can resolve other services by name. kubelet configures each Pod's `/etc/resolv.conf` to point to the CoreDNS ClusterIP.

What is a NetworkPolicy?

A NetworkPolicy is a Kubernetes resource that defines ingress and egress rules for Pod traffic using label selectors. By default, all Pod-to-Pod communication is allowed. A NetworkPolicy restricts this — for example, only allowing the claims-service to receive traffic from the API gateway, blocking everything else. NetworkPolicies are enforced by the CNI plugin (Calico, Cilium — not Flannel by default).

What is the CNI and name some plugins?

CNI (Container Network Interface) is a standard interface for configuring Pod networking. When a Pod starts, the kubelet calls the CNI plugin to assign an IP and set up routes.

Popular plugins: Calico (NetworkPolicy support, BGP routing), Cilium (eBPF-based, advanced observability), Flannel (simple overlay, no NetworkPolicy), Weave, AWS VPC CNI (assigns VPC IPs directly to Pods).

What is a headless service?

A headless service has `clusterIP: None`. Instead of a single virtual IP, DNS returns the IPs of all individual Pod endpoints. This lets clients do client-side load balancing or connect to specific Pods by name. Used with StatefulSets where each Pod has a stable DNS name like `pod-0.service.namespace.svc.cluster.local`. Kafka and Zookeeper use this pattern.

STORAGE

What is a PersistentVolume and PersistentVolumeClaim?

A PersistentVolume (PV) is a piece of storage provisioned in the cluster — an EBS volume, NFS share, or local disk — described as a Kubernetes resource with capacity and access mode.

A PersistentVolumeClaim (PVC) is a request for storage by a user or Pod. Kubernetes binds the PVC to a matching PV. The Pod mounts the PVC. This decouples how storage is provisioned from how it is consumed.

What is a StorageClass and dynamic provisioning?

A StorageClass defines a type of storage with a provisioner (e.g., `kubernetes.io/aws-ebs`) and parameters (volume type, IOPS, encryption). With dynamic provisioning, when a PVC is created referencing a StorageClass, the provisioner automatically creates a PV (e.g., an EBS volume) — no manual pre-provisioning needed. This is the standard approach in cloud environments.

Access modes — ReadWriteOnce, ReadOnlyMany, ReadWriteMany?

ReadWriteOnce (RWO): The volume can be mounted read-write by a single node. Most block storage (EBS) supports this.

ReadOnlyMany (ROX): Mounted read-only by many nodes simultaneously.

ReadWriteMany (RWX): Mounted read-write by many nodes simultaneously. Requires shared filesystems like EFS, NFS, or CephFS. Needed when multiple Pod replicas need to write to the same volume.

Reclaim policies for PersistentVolumes?

Retain: When the PVC is deleted, the PV and its data are preserved. An admin must manually reclaim it.

Delete: When the PVC is deleted, the PV and the underlying storage (e.g., EBS volume) are also deleted. Common default for dynamically provisioned volumes.

Recycle (deprecated): Basic scrub and reuse. No longer recommended.

SCHEDULING & RESOURCE MANAGEMENT

What are resource requests and limits?

Requests: The amount of CPU/memory Kubernetes guarantees for the container. The Scheduler uses requests to decide node placement.

Limits: The maximum CPU/memory the container can use. Exceeding CPU limit causes throttling. Exceeding memory limit causes the container to be OOMKilled.

Always set both. Without requests, the Scheduler cannot make good placement decisions. Without limits, a runaway container can starve other workloads on the node.

What are taints and tolerations?

A taint is applied to a node to repel Pods: `kubectl taint nodes node1 gpu=true:NoSchedule`. No Pod will be scheduled on that node unless it has a matching toleration in its spec.

Use case: Dedicate GPU nodes only for ML workloads. Taint the GPU nodes. Only ML Pods carry the toleration.

Taint effects: NoSchedule (won't schedule new Pods), PreferNoSchedule (avoid if possible), NoExecute (evict existing Pods and prevent new ones).

What are node affinity and pod affinity rules?

Node affinity: Constrains which nodes a Pod can be scheduled on based on node labels.

`requiredDuringSchedulingIgnoredDuringExecution` is hard (like nodeSelector but more expressive). `preferredDuringScheduling...` is soft.

Pod affinity: Schedule a Pod close to (or away from) other Pods matching a label selector. Anti-affinity spreads replicas across zones or nodes for HA — e.g., don't put two replicas of the same Deployment on the same node.

QoS classes — Guaranteed, Burstable, BestEffort?

Guaranteed: requests == limits for all containers. Highest priority, never throttled under normal conditions.

Burstable: requests < limits (or only requests set). Can use more resources when available but may be throttled or OOMKilled under pressure.

BestEffort: No requests or limits set. First to be evicted when the node is under memory pressure.

For production workloads, target Guaranteed or Burstable with well-tuned values.

SCALING & AVAILABILITY

How does the Horizontal Pod Autoscaler work?

HPA watches a metric (CPU utilization, memory, or custom metrics via Prometheus Adapter) and adjusts the replica count of a Deployment or StatefulSet. It queries the Metrics Server every 15 seconds (default), computes the desired replicas as

$\text{currentReplicas} * (\text{currentMetric} / \text{targetMetric})$, and scales up or down within defined min/max bounds.

HPA vs VPA?

HPA scales horizontally — adds or removes Pod replicas. Best for stateless services where you can run multiple instances.

VPA scales vertically — adjusts CPU/memory requests and limits of existing containers. Requires Pod restarts to apply changes (in most modes). Use VPA for workloads that cannot be easily replicated or to right-size resource requests.

They can conflict if both manage the same resource, so combining them requires care — typically use HPA for replica scaling and VPA in recommendation-only mode for sizing guidance.

What is a PodDisruptionBudget?

A PDB defines the minimum number (or percentage) of Pods that must remain available during voluntary disruptions — node drains, cluster upgrades, rolling updates. For example, `minAvailable: 2` ensures at least 2 replicas of a service are always running even when a node is being drained. Without a PDB, a drain could evict all Pods of a service simultaneously causing downtime.

Rolling update vs Recreate strategy?

Rolling update (default): Gradually replaces old Pods with new ones. `maxUnavailable` and `maxSurge` control the pace. Zero downtime but both old and new versions run simultaneously for a period.

Recreate: Terminates all old Pods first, then starts new ones. Causes downtime but ensures only one version runs at a time. Use when the new version is incompatible with the old (schema changes, singleton processes).

Blue-green and canary deployments?

Blue-green: Run two full environments (blue = current, green = new). Switch the Service selector from blue to green after validation. Instant cutover with easy rollback by switching back. Requires double the resources.

Canary: Route a small percentage of traffic (e.g., 5%) to the new version alongside the old. Gradually shift traffic as confidence grows. Implemented with multiple Deployments and weighted Ingress rules, or a service mesh (Istio).

SECURITY

What is RBAC in Kubernetes?

Role-Based Access Control restricts who can do what in the cluster.

Role: Defines permissions (verbs like get, list, create, delete) on resources within a namespace.

ClusterRole: Same but cluster-wide (across all namespaces or for cluster-scoped resources like nodes).

RoleBinding: Binds a Role to a user, group, or ServiceAccount within a namespace.

ClusterRoleBinding: Binds a ClusterRole cluster-wide.

Example: A developer's ServiceAccount gets a Role allowing `get/list/watch` on Pods in the dev namespace but cannot delete or access prod.

What is a ServiceAccount?

A ServiceAccount is an identity for processes running inside Pods. When a Pod needs to call the Kubernetes API (e.g., an operator reading CRDs, a CI tool creating Deployments), it authenticates using its ServiceAccount token. By default, every Pod gets the **default** ServiceAccount — which should have minimal permissions. Always create dedicated ServiceAccounts with least-privilege RBAC for workloads that need API access.

What are Pod Security Standards?

Kubernetes defines three policy levels:

Privileged: No restrictions. For system-level workloads.

Baseline: Prevents known privilege escalations — no hostPID, hostNetwork, privileged containers, or dangerous capabilities.

Restricted: Heavily hardened — must run as non-root, drop all capabilities, use seccomp profiles, disallow privilege escalation.

Enforced via Pod Security Admission controller on namespaces with labels like **pod-security.kubernetes.io/enforce: restricted**.

OBSERVABILITY

How do you view logs for a Pod?

```
# Current logs
kubectl logs <pod-name> -n <namespace>

# Previous crashed container
kubectl logs <pod-name> --previous -n <namespace>

# Specific container in a multi-container Pod
kubectl logs <pod-name> -c <container-name> -n <namespace>

# Stream live logs
kubectl logs -f <pod-name> -n <namespace>
```

Liveness, readiness, and startup probes?

Liveness probe: Checks if the container is alive. If it fails, kubelet restarts the container. Use for detecting deadlocks — the app is running but not making progress.

Readiness probe: Checks if the container is ready to accept traffic. If it fails, the Pod is removed from the Service endpoints — traffic stops going to it. Use during startup warmup or when the app temporarily can't serve.

Startup probe: Gives slow-starting containers time to initialize before liveness checks begin. If defined, liveness and readiness probes don't start until the startup probe succeeds. Prevents premature restarts of apps with long boot times.

How do you monitor a Kubernetes cluster?

The standard stack: Prometheus scrapes metrics from the Metrics Server, kube-state-metrics (cluster object state), node-exporter (node-level metrics), and application `/metrics` endpoints. Grafana visualizes dashboards. Alertmanager sends alerts to Slack, PagerDuty, etc.

For logs: Fluentd or Fluent Bit as DaemonSets ship logs to Elasticsearch (EFK stack) or CloudWatch/Loki.

For traces: OpenTelemetry SDKs in apps export traces to Jaeger or Tempo.

HELM

What is Helm and what problem does it solve?

Helm is the package manager for Kubernetes. Without Helm, deploying an application means managing many separate YAML manifests with no way to template, version, or reuse them. Helm packages everything into a chart — a collection of templated manifests with default values. You install a chart with one command, override values per environment, and roll back releases with one command.

What is values.yaml and how do you override values?

`values.yaml` is the default configuration file for a Helm chart. Templates reference values using `{{ .Values.key }}`. You override values at install/upgrade time:

```
# Override with a file
helm upgrade myapp ./chart -f prod-values.yaml
```

```
# Override individual values
helm upgrade myapp ./chart --set image.tag=v2.0.0 --set
replicaCount=3
```

What are Helm hooks?

Helm hooks let you run Jobs or other resources at specific points in the release lifecycle — `pre-install`, `post-install`, `pre-upgrade`, `post-upgrade`, `pre-delete`, `pre-rollback`. Common uses: run database migrations before a new version goes live (`pre-upgrade` hook), send a Slack notification after a successful deploy (`post-install` hook), clean up resources before deletion.

CI/CD & GITOPS

What is GitOps?

GitOps is a deployment model where Git is the single source of truth for cluster state. All Kubernetes manifests or Helm charts are stored in a Git repo. A GitOps agent (ArgoCD, Flux) continuously watches the repo and reconciles the cluster state to match it. Deployments happen by merging a PR — no direct kubectl access to production. Benefits: full audit trail, easy rollback (revert the commit), drift detection.

What is ArgoCD?

ArgoCD is a declarative GitOps continuous delivery tool for Kubernetes. It runs in the cluster, watches a Git repo, and automatically syncs the cluster state with the desired state in Git. It shows a live diff between desired and actual state, supports multiple sync strategies (automatic, manual), handles Helm charts and Kustomize, and provides a visual UI and RBAC for access control.

How do you handle secrets in a GitOps workflow?

Never commit raw Secrets to Git. Options:

Sealed Secrets: Encrypt Secrets with a cluster-specific key using `kubeseal`. The encrypted SealedSecret is safe to commit. The Sealed Secrets controller decrypts it in the cluster.

External Secrets Operator: Syncs secrets from AWS Secrets Manager, Vault, or GCP Secret Manager into Kubernetes Secrets. The Git repo only holds a reference, not the value.

HashiCorp Vault + Vault Agent: Inject secrets directly into Pods as files or environment variables via a sidecar.

TROUBLESHOOTING

Pod stuck in Pending state — how do you debug?

```
kubectl describe pod <pod-name> -n <namespace>
```

Look at the Events section at the bottom. Common causes:

Insufficient resources: `0/3 nodes are available: 3 Insufficient cpu`. Fix: scale the cluster or reduce requests.

No matching nodes: Taints without tolerations, nodeSelector mismatch, node affinity rules not satisfied.

PVC not bound: The Pod is waiting for a PersistentVolumeClaim to be bound to a PV. Check `kubectl get pvc`.

Image pull issue: Not Pending but ImagePullBackOff — check image name and registry credentials.

Pod in CrashLoopBackOff — causes and fixes?

CrashLoopBackOff means the container is starting, crashing, and Kubernetes is restarting it with exponential back-off.

Causes and fixes:

Application error on startup — check `kubectl logs <pod> --previous` for the stack trace.

Wrong command/entrypoint in the container spec — verify the image's CMD and the Pod spec.

Missing environment variable or config — the app fails because a required env var or ConfigMap key is absent.

Failed liveness probe that's too aggressive — the probe kills the container before it finishes starting. Add a startupProbe.

OOMKilled — memory limit too low. Check `kubectl describe pod` for `OOMKilled` in the last state.

Pod is Running but not receiving traffic?

Check in this order:

1. `kubectl get endpoints <service-name>` — if empty, the Service selector does not match any Pod labels.
 2. `kubectl describe service <name>` — verify the selector matches the Pod's labels exactly.
 3. Check readiness probe — if it's failing, the Pod is excluded from endpoints. `kubectl describe pod` shows readiness probe failures.
 4. Check NetworkPolicy — a policy may be blocking ingress traffic to the Pod's namespace or labels.
 5. `kubectl exec` into another Pod and `curl` the service directly to isolate the issue.
-

Node shows NotReady — steps to diagnose?

```
kubectl describe node <node-name>
# Look at Conditions and Events

# SSH into the node
systemctl status kubelet
journalctl -u kubelet -n 100

# Check resource pressure
kubectl top node <node-name>
```

Common causes: kubelet crashed or failed to start, disk pressure (node's disk is full), memory pressure, network partition between the node and the API Server, container runtime (containerd) not running.

How do you drain a node before maintenance?

```
# Cordon the node (prevent new Pods from being scheduled)
```

```
kubectl cordon <node-name>
```

```
# Drain (evict all Pods respecting PDBs)
```

```
kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data
```

```
# After maintenance, uncordon
```

```
kubectl uncordon <node-name>
```

--ignore-daemonsets skips DaemonSet-managed Pods (they will be recreated). **--delete-emptydir-data** deletes Pods using emptyDir volumes. PDBs are respected — drain will block if evicting a Pod would violate a PodDisruptionBudget.

How do you exec into a running container?

```
kubectl exec -it <pod-name> -n <namespace> -- /bin/sh
```

```
# If bash is available
```

```
kubectl exec -it <pod-name> -- /bin/bash
```

```
# Specific container in multi-container pod
```

```
kubectl exec -it <pod-name> -c <container-name> -- /bin/sh
```

For a minimal distroless image without a shell, use an ephemeral debug container:

```
kubectl debug -it <pod-name> --image=busybox --target=<container-name>
```

kubectl COMMANDS

Key commands reference:

```
# Force delete a stuck pod
```

```
kubectl delete pod <name> --grace-period=0 --force
```

```
# Get all resources across all namespaces
kubectl get all -A

# Rollback a deployment
kubectl rollout undo deployment/<name>
kubectl rollout undo deployment/<name> --to-revision=3

# View rollout history
kubectl rollout history deployment/<name>

# Apply vs Create vs Replace
# apply: declarative, creates or updates, safe for CI/CD
# create: imperative, fails if resource already exists
# replace: replaces the full resource, fails if it doesn't exist

# Explain a resource field
kubectl explain deployment.spec.strategy

# Edit a live resource
kubectl edit deployment <name>

# Port forward for local debugging
kubectl port-forward pod/<name> 8080:8080
kubectl port-forward svc/<name> 8080:80

# Get events sorted by time
kubectl get events --sort-by='.lastTimestamp' -n <namespace>

# Top resources
kubectl top pods -n <namespace>
kubectl top nodes
```

PRODUCTION ISSUES — Q&A

Issue 1: Service suddenly has no endpoints — all traffic returning 502/503

Symptoms: Ingress or downstream service getting 502. `kubectl get endpoints <service>` shows `<none>`.

Analysis:

```
kubectl get endpoints <service-name> -n <namespace>
kubectl describe service <service-name>
kubectl get pods -l <selector-labels> -n <namespace>
kubectl describe pod <pod-name> # check readiness probe status
```

Root causes:

- All Pods failing readiness probes — app startup issue, DB connection failure, config change.
- Service selector label mismatch after a Deployment template change.
- All Pods in CrashLoopBackOff.

Fix: Check Pod logs for the underlying error. Fix the app issue or config. If labels changed, fix the Service selector. If it's a readiness probe threshold issue, tune `failureThreshold` and `periodSeconds`.

Issue 2: OOMKilled — Pod keeps dying with exit code 137

Symptoms: `kubectl describe pod` shows `Last State: Terminated Reason: OOMKilled`. CrashLoopBackOff.

Analysis:

```
kubectl describe pod <name> # check memory limit and last state
kubectl top pod <name>      # current memory usage
```

Root causes: Memory limit set too low for the workload. Memory leak in the application. Sudden traffic spike causing high heap usage.

Fix: Increase the memory limit in the Deployment spec. If it's a leak, identify it with heap dumps or profiling. Use VPA in recommendation mode to right-size limits based on historical usage. Set JVM heap flags (`-Xmx`) to stay within container limits for Java apps.

Issue 3: Cluster Autoscaler not scaling up — Pods stuck in Pending

Symptoms: Pods in Pending for minutes. Cluster Autoscaler logs show scale-up attempts but nodes not joining.

Analysis:

```
kubectl describe pod <pending-pod>    # check events for
resource/taint issues
kubectl logs -n kube-system -l app=cluster-autoscaler
```

Root causes:

- Node group max size reached — Autoscaler cannot exceed configured maximum.
- Pod has hard affinity rules that no new node template satisfies.
- Cloud provider quota exceeded — AWS EC2 instance limit hit.
- Scale-up cooldown period still active.

Fix: Increase the node group max in the Autoscaler config. Relax affinity rules if overly strict. Request quota increase from the cloud provider.

Issue 4: Rolling update stuck — Deployment not progressing

Symptoms: `kubectl rollout status deployment/<name>` hangs. Old and new Pods both running for a long time.

Analysis:

```
kubectl rollout status deployment/<name>
kubectl describe deployment <name>
kubectl get pods -l app=<name>
kubectl describe pod <new-pod>    # check why new pod isn't ready
```

Root causes:

- New Pod failing readiness probe — new version has a bug or misconfiguration.
- Insufficient cluster resources — new Pods can't be scheduled (`maxSurge` pods can't fit).

- PodDisruptionBudget blocking the rollout.
- Image pull failure.

Fix: Check new Pod logs for errors. If the new version is broken, roll back immediately:

`kubectl rollout undo deployment/<name>`. Fix the underlying issue and redeploy.

Issue 5: Node pressure — Pods being evicted unexpectedly

Symptoms: Pods evicted with reason `The node was low on resource: memory`. New Pods are created but the cycle repeats.

Analysis:

```
kubectl describe node <node-name>    # check Conditions:
MemoryPressure/DiskPressure
kubectl get events -A | grep Evicted
kubectl top nodes
kubectl top pods -A --sort-by=memory
```

Root causes:

- BestEffort or Burstable Pods consuming more memory than expected.
- No resource limits set — some Pods are using far more than anticipated.
- Fluentd or node-level DaemonSet leaking memory.
- Disk pressure from large container logs filling `/var/log`.

Fix: Set resource requests and limits on all workloads. Implement log rotation. Identify and fix the memory-hungry workload. Add more nodes or scale the node group up. Increase the eviction threshold if safe.

Issue 6: Intermittent DNS resolution failures inside the cluster

Symptoms: Service-to-service calls randomly fail with connection errors or timeouts. Pod logs show `dial: lookup <service> on <dns>: i/o timeout`.

Analysis:

```
# Test DNS resolution from within a Pod
kubectl exec -it <pod> -- nslookup <service-name>
kubectl exec -it <pod> -- cat /etc/resolv.conf

# Check CoreDNS
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns
kubectl top pods -n kube-system -l k8s-app=kube-dns
```

Root causes:

- CoreDNS Pods under resource pressure — too few replicas or CPU limit too low.
- `ndots:5` in resolv.conf causing excessive search domain lookups.
- conntrack table full on the node causing UDP packet drops.
- Network plugin issues.

Fix: Scale CoreDNS replicas. Increase CoreDNS CPU/memory limits. Use FQDN for service calls to avoid search domain overhead. Increase the conntrack table size on nodes.

Issue 7: PVC stuck in Terminating state

Symptoms: `kubectl get pvc` shows PVC in `Terminating` but it never completes.

Analysis:

```
kubectl describe pvc <pvc-name>
kubectl get pvc <pvc-name> -o yaml # check finalizers
```

Root cause: The PVC has a `kubernetes.io/pvc-protection` finalizer that prevents deletion while a Pod is still using it. Or the Pod using the PVC is itself stuck in Terminating.

Fix:

```
# Force remove the finalizer
kubectl patch pvc <pvc-name> -p '{"metadata":{"finalizers":[]}}' --
type=merge
```

```
# Or fix the stuck Pod first
```

```
kubectl delete pod <pod-name> --grace-period=0 --force
```

Issue 8: HPA not scaling — stuck at minimum replicas despite high CPU

Symptoms: CPU utilization is high but HPA doesn't scale up. `kubectl describe hpa` shows `unknown` metrics.

Analysis:

```
kubectl describe hpa <name>
kubectl top pods -n <namespace>
kubectl get apiservice v1beta1.metrics.k8s.io
```

Root causes:

- Metrics Server not installed or crashing.
- Resource requests not set on the Pod — HPA computes utilization as `actual / requested`. Without requests, it can't compute a percentage.
- Custom metrics pipeline (Prometheus Adapter) misconfigured.

Fix: Install/fix the Metrics Server. Set CPU requests on the Deployment. Verify the metrics API is available: `kubectl get --raw /apis/metrics.k8s.io/v1beta1/pods`.

Issue 9: Secret or ConfigMap change not reflecting in running Pods

Symptoms: You updated a ConfigMap but the application is still using old values.

Analysis: Kubernetes does not automatically restart Pods when a ConfigMap or Secret changes.

If mounted as a volume: The file will eventually update (within `syncPeriod`, ~60s) but the app must re-read the file. Many apps require a restart to pick up config changes.

If injected as environment variable: Will never update — environment variables are set at container start time. A Pod restart is required.

Fix: Trigger a rolling restart:

```
kubectl rollout restart deployment/<name>
```

For production, use a config-reload sidecar (like Reloader by Stakater) that watches ConfigMaps/Secrets and triggers restarts automatically when they change.

Issue 10: etcd high latency or disk I/O bottleneck

Symptoms: API Server slow, `kubectl` commands time out, cluster-wide slowness. etcd logs show `took too long` warnings.

Analysis:

```
# Check etcd health
kubectl exec -it etcd-<node> -n kube-system -- etcdctl endpoint
health \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Check etcd metrics, disk latency
```

Root causes: etcd's underlying disk is too slow (spinning disk instead of SSD). etcd cluster has too many objects — too many Events, old revisions, or large Secrets. Noisy neighbor on the same node.

Fix: Run etcd on fast SSDs (NVMe recommended). Compact and defragment etcd regularly:

```
etcdctl defrag --endpoints=<endpoints>
etcdctl compact <revision>
```

Limit Event retention with `--event-ttl` on the API Server. Dedicate nodes to etcd.

Issue 11: Ingress returning 404 for all routes

Symptoms: All requests to the Ingress return 404. Service and Pods are healthy.

Analysis:

```
kubectl describe ingress <name>
kubectl get ingress <name> -o yaml
kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx
```

Root causes:

- Ingress class annotation missing or wrong — Ingress controller ignores the resource.
- Path type mismatch — **Prefix** vs **Exact** vs **ImplementationSpecific**.
- Ingress in wrong namespace — Ingress must be in the same namespace as the backend Service.
- Ingress controller not deployed or crashing.

Fix: Verify **ingressClassName** matches the deployed controller. Check path and pathType. Confirm the backend Service name and port match exactly.

Issue 12: Pod-to-Pod communication failing across nodes

Symptoms: Pods on the same node can communicate, but cross-node Pod calls fail or time out.

Analysis:

```
# Get Pod IPs and node placement
kubectl get pods -o wide

# Test connectivity
kubectl exec -it <pod-a> -- curl http://<pod-b-ip>:<port>

# Check CNI plugin status
kubectl get pods -n kube-system | grep -i cni
```

Root causes:

- CNI plugin misconfigured or crashed — Flannel/Calico not running on all nodes.
- Security group rules on cloud VMs blocking Pod CIDR traffic between nodes.

- Firewall rules on the OS blocking the overlay network ports.
- Node-level iptables rules corrupted.

Fix: Restart the CNI DaemonSet Pods. Verify cloud security groups allow Pod CIDR traffic between nodes. Check iptables rules and flush if corrupted.

This covers the full set of Kubernetes interview questions with answers and production issue analysis. Let me know if you want to go even deeper on any specific area — scenario walkthroughs, architecture diagrams, or mock interview Q&A rounds.

6/5/2026, 6:27:39 AM



Any other issues we can expect from Kubernetes based application and how do we analyze and fix it

6/5/2026, 6:28:40 AM



Here are additional real-world production issues you can expect from Kubernetes-based applications, beyond what was covered.

APPLICATION-LEVEL ISSUES

Issue: Graceful shutdown not working — requests dropping during deployments

Symptoms: During rolling updates, some in-flight HTTP requests get connection reset errors. Clients see 502 or broken pipe errors for a few seconds during every deployment.

Why it happens: When Kubernetes terminates a Pod, it sends SIGTERM to the container and simultaneously removes the Pod from the Service endpoints. But there is a race condition — kube-proxy and Ingress controllers take a few seconds to propagate the endpoint removal. During that window, traffic still arrives at the terminating Pod, which may have already stopped accepting connections.

Analysis:

```
kubectl describe pod <terminating-pod>
# Check terminationGracePeriodSeconds
# Check if app handles SIGTERM

kubectl get deployment <name> -o yaml | grep -A5 lifecycle
kubectl get deployment <name> -o yaml | grep terminationGracePeriod
```

Fix: Two changes together solve this completely.

First, add a preStop hook to delay shutdown by a few seconds, giving the load balancer time to drain:

```
lifecycle:
  preStop:
    exec:
      command: ["sh", "-c", "sleep 5"]
```

Second, ensure your application handles SIGTERM — it should stop accepting new connections, finish in-flight requests, then exit cleanly. Set

`terminationGracePeriodSeconds` to be longer than the max request duration:

```
spec:
  terminationGracePeriodSeconds: 60
```

Issue: Spring Boot application taking too long to start — readiness probe failing and causing CrashLoopBackOff

Symptoms: App works fine locally but in Kubernetes it enters CrashLoopBackOff. Logs show the app was still starting when it got killed.

Analysis:

```
kubectl describe pod <name>
# Events show: Readiness probe failed, Liveness probe failed,
killing container
kubectl logs <pod> --previous
# Logs show app not fully started yet
```

Root cause: Liveness probe fires before the app finishes initializing. Failure threshold exceeded so kubelet kills the container, which triggers CrashLoopBackOff.

Fix: Add a startupProbe that gives the app sufficient time to start. Liveness and readiness probes are suspended until the startup probe succeeds:

```
startupProbe:
  httpGet:
    path: /actuator/health
    port: 8080
  failureThreshold: 30      # 30 * 10s = 5 minutes max startup time
```



```
periodSeconds: 10

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 0
  periodSeconds: 10

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  periodSeconds: 5
```

Issue: Application connects to stale Pod IP after Pod restart

Symptoms: After a Pod restarts (new IP assigned), a client microservice keeps getting connection refused. It recovers only after the client Pod is also restarted.

Root cause: The client application cached the old Pod IP from a DNS lookup and never refreshed it. This is a DNS TTL caching problem — the JVM or HTTP client ignores DNS TTL and holds the resolved IP indefinitely.

Analysis:

```
# Confirm Pod IP changed after restart
kubectl get pod <name> -o wide

# Old IP vs new IP

# Check if service endpoints updated
kubectl get endpoints <service-name>
```

Fix: For Java/Spring Boot apps, configure the JVM DNS cache TTL:

```
# In JVM options
-Dnetworkaddress.cache.ttl=10
```

```
-Dnetworkaddress.cache.negative.ttl=0
```

Better: Always connect to the Kubernetes Service DNS name, never to Pod IPs directly. Services abstract the IP and always route to healthy endpoints.

Issue: Kafka consumer Pods causing duplicate processing after restart

Symptoms: After a Pod crash and restart, some messages are processed twice.

Downstream data has duplicate records.

Root cause: The consumer committed offsets only periodically (auto-commit). On crash, uncommitted offsets for already-processed messages were reprocessed after restart. Kubernetes restarts the Pod but Kafka has no knowledge of partial processing.

Analysis:

```
# Check consumer group lag and offset positions
kubectl exec -it <kafka-pod> -- kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --describe --group <consumer-group>
```

Fix: Disable auto-commit. Use manual offset commit after successful processing and downstream acknowledgment. Implement idempotent consumers — use a deduplication key (messageId, correlationId) stored in Redis or DB to detect and skip already-processed messages. For critical flows, use Kafka's exactly-once semantics with transactional producers and consumers.

Issue: Init Container failing silently — Pod stuck in Init state indefinitely

Symptoms: Pod shows `Init:0/1` for a long time. No obvious error in `kubectl get pods`.

Analysis:

```
kubectl describe pod <name>
# Check Init container status in the output

kubectl logs <pod-name> -c <init-container-name>
# -c flag specifies the init container by name
```

Common causes: Init container is waiting for a dependency (DB, external service) that is down or unreachable. It loops forever with no timeout. DNS resolution failing inside the init container.

Fix: Add a timeout to the init container's wait logic. Use `curl --max-time` or `timeout` command in shell scripts. Ensure the service the init container depends on has a proper readiness check of its own. Never let init containers block indefinitely in production.

NETWORKING ISSUES

Issue: Service mesh sidecar (Istio/Envoy) intercepting health check traffic causing failures

Symptoms: After enabling Istio, liveness and readiness probes start failing even though the app is healthy. mTLS errors in Envoy logs.

Root cause: Istio's Envoy sidecar intercepts all traffic including kubelet's HTTP health check probes. If mTLS is enforced and kubelet does not present a certificate, the probe fails.

Analysis:

```
kubectl logs <pod-name> -c istio-proxy
# Look for TLS handshake errors or 503s on probe paths

kubectl describe pod <name>
# Check probe configuration
```

Fix: Annotate the Pod to exclude probe ports from Istio interception:

```
annotations:
  traffic.sidecar.istio.io/excludeInboundPorts: "8080"
```

Or use exec-based probes that run inside the container and bypass the sidecar, or switch to gRPC probes. In newer Istio versions, use `rewriteAppHTTPProbers: true` in the mesh config which rewrites probes automatically.

Issue: kube-proxy iptables rules causing connection timeouts under high load

Symptoms: Under high traffic, some connections time out even though target Pods are healthy. Intermittent 504s. The problem worsens with more Services.

Root cause: kube-proxy in iptables mode creates $O(n)$ rules per Service. With hundreds of Services, iptables traversal becomes slow. Under high connection rate, connection tracking table fills up causing drops.

Analysis:

```
# Check number of iptables rules
iptables -L | wc -l

# Check conntrack table usage
cat /proc/sys/net/netfilter/nf_conntrack_count
cat /proc/sys/net/netfilter/nf_conntrack_max

# Check kube-proxy mode
kubectl get configmap kube-proxy -n kube-system -o yaml | grep mode
```

Fix: Switch kube-proxy to IPVS mode, which uses hash tables and scales much better:

```
# In kube-proxy ConfigMap
mode: "ipvs"
```

Increase the conntrack table size on all nodes. Consider migrating to Cilium with eBPF-based networking which replaces kube-proxy entirely and provides much better performance at scale.

Issue: Intermittent 503 from Ingress — NGINX upstream connection errors

Symptoms: Occasional 503 errors in NGINX Ingress logs: `upstream prematurely closed connection`. Happens randomly but more frequently during deployments.

Root cause: NGINX keeps persistent (keepalive) connections to upstream Pods. When a Pod terminates, NGINX's cached connection to it becomes stale. The next request on that connection gets a connection reset.

Analysis:

```
kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx --tail=200
# Look for upstream errors and which Pod IP caused them

kubectl describe ingress <name>
```

Fix: Configure upstream keepalive with proper timeout settings in the Ingress annotations:

```
annotations:
  nginx.ingress.kubernetes.io/upstream-keepalive-connections: "0" #
  disable keepalive
  # OR tune keepalive timeout to be shorter than Pod termination
  time
```

Also, ensure Pods use the preStop sleep hook (covered above) so the connection drains before the Pod exits.

Issue: ExternalDNS not creating Route53 records for new Ingress

Symptoms: New Ingress is deployed but DNS record not created in Route53. Service unreachable from the internet.

Analysis:

```
kubectl logs -n kube-system -l app=external-dns --tail=100
# Look for permission errors or annotation mismatches

kubectl describe ingress <name>
# Check annotations match ExternalDNS configuration
```

Root causes: IAM role attached to ExternalDNS ServiceAccount lacks Route53 permissions. Ingress annotation `external-dns.alpha.kubernetes.io/hostname` missing or misspelled. ExternalDNS is filtering by namespace or class that doesn't include this Ingress.

Fix: Verify the IRSA (IAM Role for ServiceAccount) has `route53:ChangeResourceRecordSets` and `route53:ListHostedZones` permissions. Add the correct hostname annotation to the Ingress. Check ExternalDNS `--domain-filter` and `--source` flags match your setup.

STORAGE ISSUES

Issue: Pod failing to mount EBS volume — stuck in ContainerCreating

Symptoms: Pod stays in `ContainerCreating` for minutes. `kubectl describe pod` shows volume attachment errors.

Analysis:

```
kubectl describe pod <name>
# Events: Unable to attach or mount volumes

kubectl describe pvc <pvc-name>
# Check bound status and volume ID

# Check volume attachment on AWS
aws ec2 describe-volumes --volume-ids <vol-id>
```

Root causes: EBS volume is still attached to the old node (the previous Pod's node). EBS volumes are AZ-scoped — if the Pod was rescheduled to a different AZ, it cannot attach the volume. AWS has a per-node EBS volume attachment limit (varies by instance type).

Fix: EBS volumes are zonal. Use node affinity to pin StatefulSet Pods to the correct AZ, or use topology-aware volume provisioning so the volume and Pod always land in the same AZ. For multi-AZ stateful workloads, use EFS (ReadWriteMany) instead of EBS (ReadWriteOnce).

Issue: Disk pressure on nodes due to container image and log accumulation

Symptoms: Nodes show `DiskPressure=True`. Pods are evicted. New Pods cannot be scheduled. Node disk usage at 90%+.

Analysis:

```
kubectl describe node <name> | grep -A5 Conditions
kubectl describe node <name> | grep -A5 "DiskPressure"

# SSH into node
df -h
```

```
du -sh /var/lib/docker/overlay2/* # or containerd  
du -sh /var/log/pods/*
```

Root causes: Old container image layers accumulating. Container logs not rotated — a chatty container generating gigabytes of logs. Large emptyDir volumes not cleaned up. Leftover completed Job Pod logs.

Fix: Configure container log rotation on the node's container runtime. Implement log rotation in containerd config:

```
"max-size": "100m"  
"max-file": "3"
```

Enable the kubelet's image garbage collection:

```
imageGCHighThresholdPercent: 85  
imageGCLowThresholdPercent: 80
```

Use centralized logging (Fluent Bit) to ship logs externally and reduce local log size.

RBAC & SECURITY ISSUES

Issue: Pods unable to call Kubernetes API — 403 Forbidden errors

Symptoms: An operator, controller, or app that calls the Kubernetes API (list Pods, read ConfigMaps, etc.) gets 403 Forbidden errors in logs.

Analysis:

```
kubectl logs <pod-name>  
# Look for 403 errors with API resource and verb  
  
# Check what ServiceAccount the Pod uses  
kubectl get pod <name> -o yaml | grep serviceAccountName  
  
# Check what RBAC is bound to that ServiceAccount  
kubectl get rolebindings,clusterrolebindings -A | grep  
<serviceaccount-name>
```

```
# Test permissions directly
```

```
kubectl auth can-i list pods --as=system:serviceaccount:<namespace>:  
<sa-name>
```

Fix: Create the missing Role/ClusterRole with the required permissions and bind it to the ServiceAccount:

```
apiVersion: rbac.authorization.k8s.io/v1  
kind: Role  
metadata:  
  name: pod-reader  
  namespace: default  
rules:  
- apiGroups: [""]  
  resources: ["pods", "configmaps"]  
  verbs: ["get", "list", "watch"]  
---  
kind: RoleBinding  
apiVersion: rbac.authorization.k8s.io/v1  
metadata:  
  name: pod-reader-binding  
subjects:  
- kind: ServiceAccount  
  name: my-app-sa  
  namespace: default  
roleRef:  
  kind: Role  
  name: pod-reader  
  apiGroup: rbac.authorization.k8s.io
```

Issue: Secrets exposed in environment variables visible in Pod spec

Symptoms: During a security audit, raw secret values are visible in `kubectl describe pod` output or in CI/CD logs where `env` is printed.

Root cause: Secrets injected as environment variables are visible in the Pod spec and in process environment listings (`/proc/<pid>/environ`). Any user with Pod describe access can read them.

Fix: Mount Secrets as files in a volume instead of environment variables:

```
volumeMounts:
- name: db-secret
  mountPath: /etc/secrets
  readOnly: true
volumes:
- name: db-secret
  secret:
    secretName: db-credentials
```

The app reads the file at runtime. Files are not visible in Pod describe output. Additionally, enable Secret encryption at rest in etcd using a KMS provider (AWS KMS). Use the External Secrets Operator to pull secrets from Vault or AWS Secrets Manager rather than storing them in etcd at all.

PERFORMANCE ISSUES

Issue: Java application throttled despite low CPU utilization — JVM performance degraded

Symptoms: Spring Boot app shows high latency and GC pauses. `kubectl top pod` shows CPU well below the limit. Thread dumps show long GC stop-the-world events.

Root cause: The JVM sees the node's total CPU count (e.g., 64 cores) and configures the GC thread count accordingly — creating far more GC threads than the container's CPU limit allows. All those threads fight for 1 CPU, causing severe throttling and GC pauses.

Analysis:

```
kubectl exec -it <pod> -- java -XshowSettings:all -version 2>&1 |
grep -i cpu
kubectl exec -it <pod> -- cat /sys/fs/cgroup/cpu/cpu.cfs_quota_us
```

Fix: Use container-aware JVM flags (JDK 11+ handles this automatically, but be explicit):

```
JAVA_OPTS="-XX:+UseContainerSupport \
           -XX:MaxRAMPercentage=75.0 \
           -XX:ActiveProcessorCount=2"
```

Set CPU requests equal to limits (Guaranteed QoS) to prevent CPU throttling in the CFS scheduler. This is a very common production issue with Java on Kubernetes.

Issue: Prometheus scraping causing high CPU on application Pods

Symptoms: During Prometheus scrape intervals, application CPU spikes. Latency increases every 15 seconds in sync with the scrape interval.

Root cause: The `/metrics` endpoint is computationally expensive — generating metrics requires iterating over large data structures, computing histograms, or holding locks. Under a 15-second scrape interval with many instances, this becomes a constant overhead.

Analysis:

```
kubectl top pods -n <namespace>
# Observe CPU spikes at regular intervals

# Check Prometheus scrape config
kubectl get servicemonitor <name> -o yaml | grep interval
```

Fix: Increase the scrape interval for expensive metrics endpoints to 30s or 60s. Optimize the metrics endpoint — cache computed values, use lazy evaluation. Separate cheap process metrics from expensive business metrics with different scrape intervals. Use Prometheus recording rules to pre-compute expensive queries instead of computing at scrape time.

Issue: Too many open file descriptors — Pod crashing with "too many open files"

Symptoms: Application logs show `java.io.IOException: Too many open files` or `EMFILE` error. Pod crashes.

Root cause: The container's file descriptor limit (`ulimit -n`) is too low. Kubernetes inherits node-level `nofile` limits. A connection-heavy service (database connections, Kafka clients, Netty connections) can exhaust the default limit of 1024 or 4096.

Analysis:

```
kubectl exec -it <pod> -- ulimit -n
kubectl exec -it <pod> -- cat /proc/1/limits | grep "open files"
kubectl exec -it <pod> -- ls /proc/1/fd | wc -l # current open FDs
```

Fix: Increase the file descriptor limit via Pod security context:

```
securityContext:
  sysctls:
    - name: fs.file-max
      value: "65535"
```

Or via container security context using `ulimit` in the container start command. For AWS Fargate or restricted environments, tune the connection pool sizes in the application to stay within the limit.

CLUSTER OPERATIONS ISSUES

Issue: kubectl commands timing out — API Server overloaded

Symptoms: `kubectl get pods` is slow or times out. All cluster operations are sluggish. API Server CPU is high.

Analysis:

```
kubectl get --raw /metrics | grep apiserver_request_duration

# Check request rate
kubectl get --raw /metrics | grep apiserver_request_total

# Check etcd latency
kubectl get --raw /metrics | grep etcd_request_duration
```

Root causes: Too many controllers or operators making excessive API watch/list calls with no label selectors (listing all Pods across all namespaces). Poorly written custom operators doing polling instead of using informers/watches. Overly verbose audit logging. Many concurrent kubectl users.

Fix: Use label selectors in all API calls. Replace polling with watch-based informers in custom operators. Rate-limit access with API Priority and Fairness (APF) in Kubernetes 1.20+. Scale the API Server horizontally (multiple replicas behind a load balancer). Tune etcd performance as described earlier.

Issue: Cluster upgrade causing unexpected Pod evictions and downtime

Symptoms: During a Kubernetes version upgrade, multiple services experience brief downtime. PodDisruptionBudgets were defined but still disruption occurred.

Analysis:

```
kubectl get events --sort-by='.lastTimestamp' -A | grep -i evict
kubectl get pdb -A
kubectl describe pdb <name>
```

Root causes: PDB `minAvailable` was set to 1 but the Deployment only had 1 replica — the PDB blocks the drain but the node drain still eventually times out and force-evicts. PDB selector not matching Pod labels (typo). All replicas on the same node being drained simultaneously.

Fix: Always run at least 2 replicas for any service that needs HA. Ensure PDB selectors match your Pod labels exactly. Use `podAntiAffinity` to spread replicas across nodes so a single node drain never takes down all instances. Test drain procedures in staging before production upgrades.

Issue: Orphaned resources accumulating — namespaces full of completed Jobs and old ReplicaSets

Symptoms: `kubectl get all -n <namespace>` returns hundreds of completed Jobs, old ReplicaSets with 0 desired replicas, and old Pods in Completed state. Etcd size growing over time.

Fix: Set `ttlSecondsAfterFinished` on Jobs for automatic cleanup:

```
spec:
  ttlSecondsAfterFinished: 3600 # clean up 1 hour after completion
```

Set `revisionHistoryLimit` on Deployments to limit old ReplicaSet retention:

spec:

```
revisionHistoryLimit: 3 # keep only last 3 ReplicaSets
```

Use a periodic CronJob or a tool like `kubectrl-janitor` to clean up stale resources. Monitor etcd size and set up alerts before it hits quota limits.

Issue: Webhook admission controller down — all new Pod creations failing

Symptoms: Suddenly no new Pods can be created across the entire cluster. `kubectrl run` or Deployment scale-up fails with webhook errors.

Analysis:

```
kubectrl get mutatingwebhookconfigurations
kubectrl get validatingwebhookconfigurations

# Check if webhook service is healthy
kubectrl get pods -n <webhook-namespace>
kubectrl describe mutatingwebhookconfiguration <name>

# Look at failurePolicy
```

Root cause: A MutatingWebhookConfiguration (Istio injection, OPA Gatekeeper, Vault agent injector) has `failurePolicy: Fail` and its backing service is down. Every Pod creation request hits the webhook, gets no response, and fails.

Fix (emergency): Temporarily change `failurePolicy` to `Ignore` or delete the webhook configuration to restore cluster operations:

```
kubectrl delete mutatingwebhookconfiguration <name>
```

Long-term fix: Always deploy webhook services with high availability (multiple replicas, PDB). Use `namespaceSelector` to exclude `kube-system` from webhooks to prevent control plane issues. Set `failurePolicy: Ignore` for non-critical webhooks.

Issue: ImagePullBackOff — private registry authentication failing

Symptoms: Pods stuck in `ImagePullBackOff`. `kubectrl describe pod` shows `unauthorized: authentication required`.

Analysis:

```
kubectl describe pod <name>
# Events: Failed to pull image "registry/image:tag": unauthorized

kubectl get secret <registry-secret> -o yaml
# Check if imagePullSecret exists and is valid

kubectl get serviceaccount default -o yaml
# Check if imagePullSecrets is attached
```

Root causes: Docker registry credentials expired (ECR tokens expire every 12 hours).

imagePullSecret not attached to the ServiceAccount or Pod spec. Secret created in wrong namespace. ECR policy not allowing the node's IAM role.

Fix for AWS ECR: Use the **amazon-ecr-credential-helper** on nodes, or deploy the ECR credential refresher as a CronJob that recreates the pull secret every 6 hours. Better: use IRSA with ECR pull-through cache and allow the node IAM role to pull images without explicit secrets.

This covers a wide range of additional production Kubernetes issues across application behavior, networking, storage, security, performance, and cluster operations. Want me to drill into any specific area with deeper RCA walkthroughs or runbook-style resolution steps?

6/5/2026, 6:30:34 AM